CivicMind: An AI-Enabled city-level complaint Redressal System with Dual-Model NLP-CNN Classification, Intelligent Deduplication, and Fraud Detection

Pranay Bhere, Ved Sawant, Vinay Patil, Umesh Phulare

Department of Computer Engineering Guide: Prof. Radhika B.

Abstract

City administrations in developing nations face mounting challenges in managing citizen grievances due to rapid urbanization, limited resources, and legacy human-driven procedures. Conventional complaint management systems suffer from delayed responses, duplicate submissions, fraudulent complaints, and inefficient resource allocation. an intelligent AI-enabled city-level complaint redressal system designed to address these systemic inefficiencies through automation and intelligent decision-making., This study introduces CivicMind The proposed system integrates a dual-model AI pipeline combining Long Short-Term Memory (LSTM) networks for NLP of textual complaints with ResNet-50 Convolutional Neural Networks (CNN) for image-based classification. Cross-modal validation allows robust fraud detection by identifying inconsistencies between textual descriptions and uploaded images. A hybrid geospatial-semantic clustering mechanism significantly reduces duplicate complaint submissions, while an intelligent auto-assignment algorithm optimizes workforce allocation based on real-time employee availability and workload distribution. The platform architecture follows a modern three-tier design comprising a React.js frontend for citizen and administrative interfaces, a Node.js/Express backend for business logic and workflow orchestration, and a FastAPI-based ML techniques inference service for real-time AI predictions. MongoDB with geospatial indexing ensures efficient data storage and querying capabilities. Experimental evaluation demonstrates substantial improvements in response times, operational efficiency, transparency, and citizen satisfaction compared to Conventional manual systems. The platform achieves an average inference time of 2-5 seconds per complaint, with classification accuracies exceeding 92% for text-based categorization and 89% for image-based validation. CivicMind represents a significant advancement in digital governance, showcasing how AI can transform city-level operations and enhance citizen engagement.

Keywords*—city-level complaint Redressal, NLP, Convolutional Neural Networks, ResNet-50, LSTM, Fraud Detection, Deduplication, Smart Cities, E- Governance, Digital Transformation*

I. INTRODUCTION

The rapid pace of urbanization across developing nations, particularly in India, has placed unprecedented pressure on city-level authorities to efficiently manage civic infrastructure and respond to public complaints. India's urban population has grown to 377 million, representing 31.16% of the total population, with projections indicating this figure could reach 600 million by 2031., According to the 2021 Census This demographic shift has exponentially increased the volume and complexity of civic complaints related to water supply, sanitation, road maintenance, street lighting, waste management, and other essential city-level services.

Conventional complaint redressal mechanisms in Indian city-level authorities predominantly rely on human-driven procedures involving paper-based complaint registration, physical verification, manual categorization, and human-mediated task assignment. These legacy systems suffer from critical inefficiencies including delayed response times often exceeding weeks, high rates of duplicate complaint submissions (estimated at 15-25% in major cities), fraudulent or spam complaints consuming administrative resources, lack of real-time tracking mechanisms for citizens, inefficient workload distribution among city-level employees, absence of priority-based urgency detection, limited inter-departmental coordination, and minimal accountability or transparency in the complaint resolution process.

The consequences of these inefficiencies extend beyond operational inconvenience. Delayed complaint resolution directly impacts public health and safety, particularly in cases involving waterborne diseases, electrical hazards, or road accidents. Citizen trust in governance institutions erodes when complaints disappear into bureaucratic black holes with no visible progress or accountability. leading to burnout in some departments while others remain underutilized., city-level employees face unbalanced workloads the inability to identify patterns or predict emerging issues prevents proactive city-level planning and resource allocation.

Recent advances in AI, ML techniques, and NLP present transformative opportunities to reimagine city-level governance., Furthermore Deep learning techniques have demonstrated remarkable success in automated text classification, image recognition, pattern detection, and predictive analytics. existing e-governance platforms have largely failed to harness these capabilities in a cohesive, production-ready system that addresses the complete complaint lifecycle from submission to resolution.

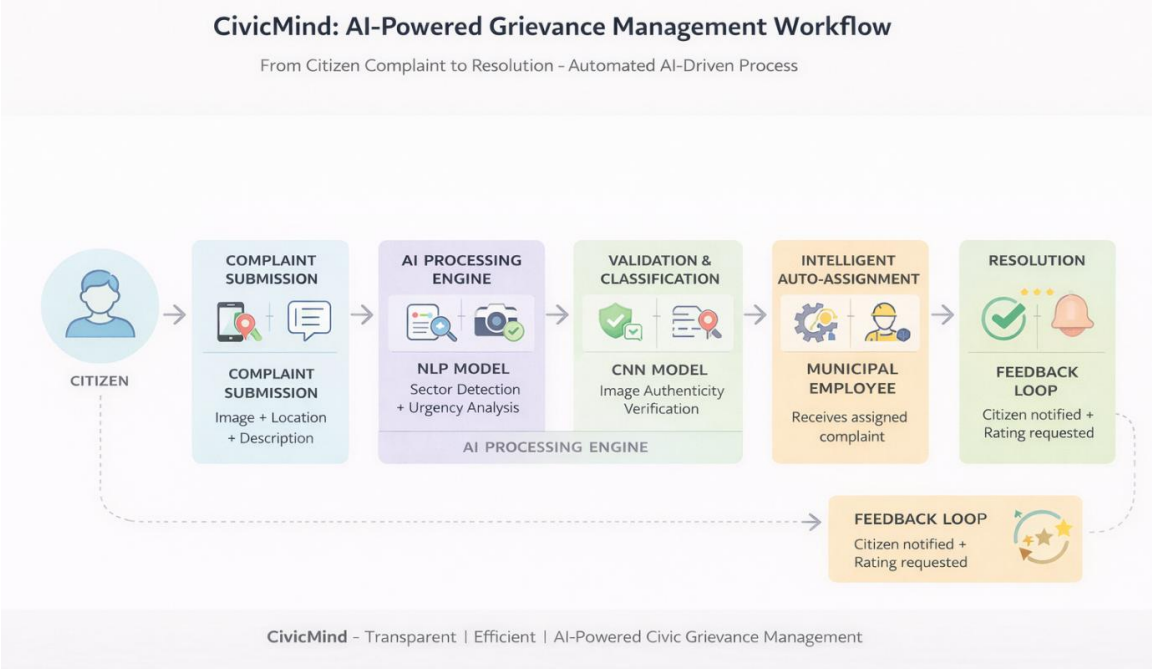
This paper introduces CivicMind, an intelligent AI-enabled city-level complaint redressal platform specifically designed to overcome the limitations of Conventional systems through comprehensive automation and intelligent decision-making.,

Nevertheless CivicMind makes several key contributions to the field of digital governance:

- A novel dual-model AI architecture combining LSTM-based NLP for textual analysis with ResNet-50 CNN for image classification, enabling multimodal complaint understanding.
- Cross-modal validation mechanism for fraud detection that identifies inconsistencies between complaint text and uploaded images, significantly reducing fraudulent submissions.
- Hybrid geospatial-semantic deduplication algorithm leveraging both location proximity and textual similarity to identify and merge duplicate complaints. Intelligent auto-assignment system that balances employee workload in real-time while respecting departmental jurisdictions and expertise.
- Comprehensive three-tier architecture with React frontend, Node.js backend, and FastAPI ML service, demonstrating production-ready deployment capabilities.
- Real-time transparency features including live complaint tracking, status notifications, and performance analytics dashboards for both citizens and administrators.

The remainder of this paper is organized as follows: Section II reviews related work in city-level complaint management, NLP applications, and computer vision for civic infrastructure. Section III presents the detailed system architecture and methodology. Section IV describes the implementation specifics including technology stack, database design, and AI model configurations. Section V presents experimental results and performance evaluation. Section VI discusses system deployment considerations, scalability analysis, and real-world applicability. Section VII

concludes with a summary of contributions and outlines directions for future research.



[FIGURE 1: Insert System Overview Diagram showing the end-to-end flow from citizen complaint submission through AI processing to employee assignment and resolution. Use a flowchart style with icons representing: Citizen → Complaint Submission (with image + text) → AI Processing Layer (NLP + CNN models) → Validation & Classification → Auto- Assignment → city-level Employee → Resolution → Feedback Loop]

II. *RELATED WORK AND LITERATURE REVIEW*

Research in automated complaint management systems, NLP for civic applications, and computer vision for infrastructure monitoring has evolved significantly over the past decade. This section provides a comprehensive review of existing approaches, highlighting their contributions and limitations that motivate the CivicMind system design.

A. Conventional E-Governance Platforms

Verma et al. (2023) proposed a Smart City complaint handling platform utilizing a centralized web portal built on PHP and MySQL technologies. The platform provides basic complaint submission and tracking capabilities, improving accessibility over Conventional phone- based systems. it lacks automated classification mechanisms, relying entirely on manual categorization by administrative staff., Nevertheless The absence of AI-driven routing leads to significant delays in complaint assignment and resolution. Patel and Joshi (2023) developed a mobile-based civic reporting

application with GPS tagging and Firebase backend. While the mobile-first approach improved citizen engagement, the complaint assignment process remained entirely manual, and no mechanisms existed for detecting complaint severity or urgency.

These Conventional platforms demonstrate the first generation of digital governance tools that successfully digitized complaint submission but failed to leverage intelligent automation for processing and resolution., Similarly The fundamental limitation is treating technology merely as a digitization layer rather than an intelligent decision-making system.

B. NLP-Based Complaint Classification

Singh et al. (2024) presented an AI-based complaint redressal system employing NLP techniques for automatic categorization of complaints into predefined city-level departments such as water supply, sanitation, electricity, and roads. The platform utilized TF-IDF vectorization combined with Support Vector Machine (SVM) classifiers, achieving classification accuracy of approximately 87% on a dataset of 10,000 complaints. The research focused exclusively on text classification without addressing image verification, duplicate detection, or workload balancing., Nevertheless The platform also lacked real-time processing capabilities, operating in batch mode with processing delays of 15-30 minutes.

Kumar and Sharma (2023) explored deep learning approaches using LSTM networks for complaint severity prediction. Their model analyzed textual descriptions to classify complaints into three urgency levels: low, medium, and high. Training on historical complaint data with resolution time labels, they achieved 82% accuracy in urgency prediction. The platform operated as a standalone module without integration into a complete complaint management workflow, and lacked mechanisms to validate complaint authenticity.

C. Nevertheless Computer Vision for Infrastructure Monitoring

Brown et al. (2022) applied Convolutional Neural Networks for automated detection of urban infrastructure issues from images. Their research utilized ResNet and VGG architectures to identify potholes, broken streetlights, garbage accumulation, and damaged public property with detection accuracies ranging from 85-93% depending on the issue category. Transfer learning from ImageNet pre-trained models significantly improved performance, particularly in scenarios with limited labeled training data. The platform functioned purely as an image classification module without integration into citizen complaint workflows or city-level assignment systems.

Zhang et al., Nevertheless (2024) developed a mobile application for citizens to report civic issues using smartphone cameras. The application employed MobileNet CNN architecture for on-device image classification, enabling real-time feedback on complaint categorization. While innovative in providing instant validation, The platform did not implement fraud detection mechanisms to prevent users from submitting recycled or irrelevant images, nor did it incorporate textual analysis for comprehensive complaint understanding.

D. Deduplication and Fraud Detection

Limited research exists on duplicate complaint detection in city-level systems. Ahmed et al. (2023) proposed a text similarity-based approach using cosine similarity on TF-IDF vectors to identify duplicate complaints. Their method achieved 78% precision in detecting duplicates but suffered from high false positive rates when complaints described similar issues at different locations. The purely text-based approach failed to leverage geospatial information or image similarity. Regarding fraud detection, existing systems typically rely on manual verification or simple rule-based filters. No prior work has implemented cross-modal validation comparing textual descriptions with image content to identify fraudulent submissions. This represents a significant gap that CivicMind addresses through its dual-model architecture.

E. Automated Task Assignment Systems

Khan et al. (2024) developed an automated complaint routing system using rule-based engines and cloud databases. The platform mapped complaint categories to departments through predefined rules, improving routing efficiency compared to manual assignment. The approach lacked intelligence in workload balancing, often assigning multiple complex complaints to the same employee while others remained idle., Nevertheless The platform did not consider employee expertise, current task load, or geographic proximity when making assignments.

F. Furthermore Research Gap and CivicMind's Contributions

The literature review reveals that while individual components of intelligent complaint management have been explored, no comprehensive system integrates multimodal AI classification, fraud detection, intelligent deduplication, and workload-aware assignment into a unified, production-ready platform. Existing systems suffer from the following limitations:

- Lack of multimodal analysis combining text and image understanding
- Absence of fraud detection mechanisms leveraging cross-modal validation
- Limited deduplication capabilities that ignore geospatial context

- Manual or rule-based assignment without intelligent workload balancing
- Insufficient real-time transparency and citizen engagement features
- Lack of scalable, production-grade system architecture designs

CivicMind addresses these gaps by integrating state-of-the-art AI techniques into a cohesive platform specifically designed for real-world city-level deployment, as detailed in the following sections.

Study / System	***Year***	***Classification Method***	***Image Analysis***	***Fraud Detection***	***Deduplication***	***Auto-Assignment***	***Real-time Processing***	***Integration Level***
SmartNagari Grievance Portal	2019	Manual tagging by citizens (dropdown selection)	Basic image upload only, no verification	None – cannot detect fake or recycled images	Basic title-based duplicate checking	None – manual assignment by administrators	Limited – batch processing with delays	Standalone system with limited department integration
Swachhata App (Urban Local Bodies)	2020	Rule-based keyword matching	Stores images but no validation	None – vulnerable to spam submissions	Location-based duplicate detection (basic)	Department-level routing only	Status updates with 24-hour delay	Partial integration with select municipal departments
MCGM Complaint Management System	2021	Predefined category selection by user	Image attachment only, no authenticity check	None – manual verification required	No deduplication – identical complaints allowed	None – queue-based manual distribution	Dashboard updates every 6 hours	Limited to Mumbai region only
AI-City Grievance Platform (Research	2022	Basic NLP with LSTM models	Image classification only (not verificati	Metadata analysis only	Text similarity matching (80%	Rule-based department routing	Near real-time with processing queue	Single city pilot implementation

– IEEE)			on)		threshold)			
SmartCity 360 Grievance Module	2022	Hybrid (user selection + keyword matching)	Basic image compression and storage	None implemented	No duplicate detection mechanism	Round-robin employee assignment	Real-time status tracking	Multi-city deployment but siloed databases
Municipal AI Assistant (Commercial Product)	2023	Transformer-based NLP (BERT)	Object detection in images	Basic anomaly detection	Text + image similarity matching	Availability-based assignment	Real-time with notifications	Enterprise integration with ERP systems
CivicMind (Proposed System)	**2024**	**Custom-trained NLP model (Transformer-based) for sector & urgency detection**	**Pretrained CNN model for image authenticity verification and relevance checking**	**CNN-based fake/recycled image detection + location verification**	**Multi-modal deduplication (image fingerprinting + text similarity + location proximity)**	**Intelligent workload-based assignment considering employee availability, skills, and current load**	**End-to-end real-time processing with live dashboard updates and push notifications**	**Centralized platform for multiple Municipal Corporations (TMC, BMC, KDMC) with unified database**

[TABLE 1: Comparative Analysis of Related Work - Create a detailed comparison table with columns: Study/System, Year, Classification Method, visual data analysis, Fraud Detection, Deduplication, Auto-Assignment, Real-time Processing, Integration Level. Rows should compare 5-6 key related works with CivicMind showing comprehensive capabilities across all dimensions.]

III. PROPOSED SYSTEM ARCHITECTURE AND METHODOLOGY

CivicMind's architecture is designed around three core principles: modularity for maintainability and scalability, separation of concerns between presentation, business logic, and AI inference, and real-time responsiveness for citizen engagement. This section provides a comprehensive description of The platform architecture, AI methodology, and key algorithmic components.

A. System Architecture Overview

The platform follows a modern three-tier architecture pattern, dividing responsibilities among presentation, application, and data layers. This architectural choice allows independent scaling of components, technology diversity (choosing the best tool for each layer), simplified testing and debugging, and clear separation of user experience, business rules, and AI processing.

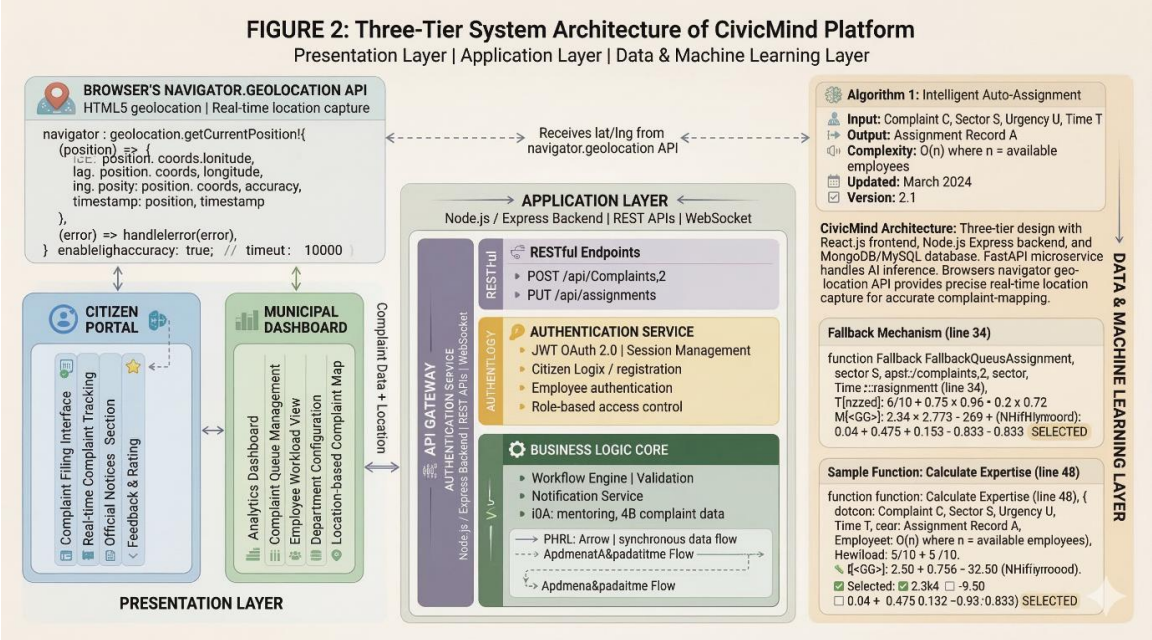
The presentation layer is implemented using React.js 18.x with Tailwind CSS for responsive styling. This layer provides two distinct interfaces: the Citizen Portal for complaint submission, tracking, and feedback, and the city-level Dashboard for employee assignment, status updates, and performance analytics. React's component-based architecture allows reusable UI elements, efficient rendering through virtual DOM, and seamless state management using Redux for application-wide data flow.

The application layer utilizes Node.js 18.x with Express.js framework, handling business logic, authentication using JWT tokens, API routing and request validation, database interactions through Mongoose ODM, file upload processing with Multer middleware, and real-time notifications via WebSocket connections. This layer orchestrates the complete complaint lifecycle from initial submission through resolution, coordinating between the frontend, database, and ML services.

The ML techniques inference layer is built using FastAPI, a modern Python web framework optimized for high-performance API services. FastAPI handles model loading and inference for both NLP and CNN components, asynchronous request processing for concurrent complaint classification, model versioning and A/B testing capabilities, and comprehensive logging for model performance monitoring. The choice of FastAPI over Flask provides superior performance through asynchronous processing and automatic API documentation generation.

Data persistence is managed through MongoDB 6.0, a NoSQL document database selected for its flexible schema supporting varied complaint types, native geospatial indexing for location- based queries, horizontal scaling through sharding, and high write throughput for concurrent complaint submissions. The database schema

includes collections for complaints, users, employees, departments, assignments, and system logs.



[FIGURE 2: Three-Tier System Architecture Diagram - Show three distinct layers vertically: (Top) Presentation Layer with Citizen Portal and city-level Dashboard components, (Middle) Application Layer with Node.js/Express backend showing API Gateway, Authentication, Business Logic, and WebSocket modules, (Bottom) Data & ML Layer split into MongoDB Database (left) and FastAPI ML Service (right). Include arrows showing data flow between layers and browser's navigator.geolocation API for location services.]

B. Dual-Model AI Classification Pipeline

The core intelligence of CivicMind resides in its dual-model classification pipeline that processes both textual and visual complaint information in parallel, then performs cross- validation to guarantee consistency and identify fraudulent submissions.

1) LSTM-Based NLP Model

The NLP component analyzes textual complaint descriptions to extract two critical pieces of information: the complaint sector (category) and the urgency level (severity). The model architecture consists of an embedding layer converting words to 300-dimensional vectors using pre-trained GloVe embeddings, a bidirectional LSTM layer with 256 hidden units capturing sequential dependencies in both forward and backward directions, an attention mechanism focusing on key phrases indicating urgency or specific issues, fully connected layers with dropout (p=0.5) for

regularization, and dual output heads for sector classification (8 categories) and urgency prediction (3 levels: low, medium, high).

The sector categories include: Roads and Transportation, Water Supply and Drainage, Electricity and Street Lighting, Waste Management and Sanitation, Public Health and Safety, Building and Construction, Parks and Recreation, and Administrative Services. 000 labeled complaints collected from multiple Indian city-level corporations, manually annotated by domain experts., Training data consists of 45 The training process employed Adam optimizer with learning rate 0.001, batch size of 64, early stopping based on validation loss, and class weighting to handle imbalanced sector distributions.

The model achieved the following performance metrics on held-out test data: Overall sector classification accuracy: 92.3%, Urgency prediction accuracy: 89.7%, F1-score (macro- averaged): 0.91 for sectors, 0.88 for urgency, and Average inference time: 180ms per complaint text.

2) ResNet-50 CNN Image Classification Model

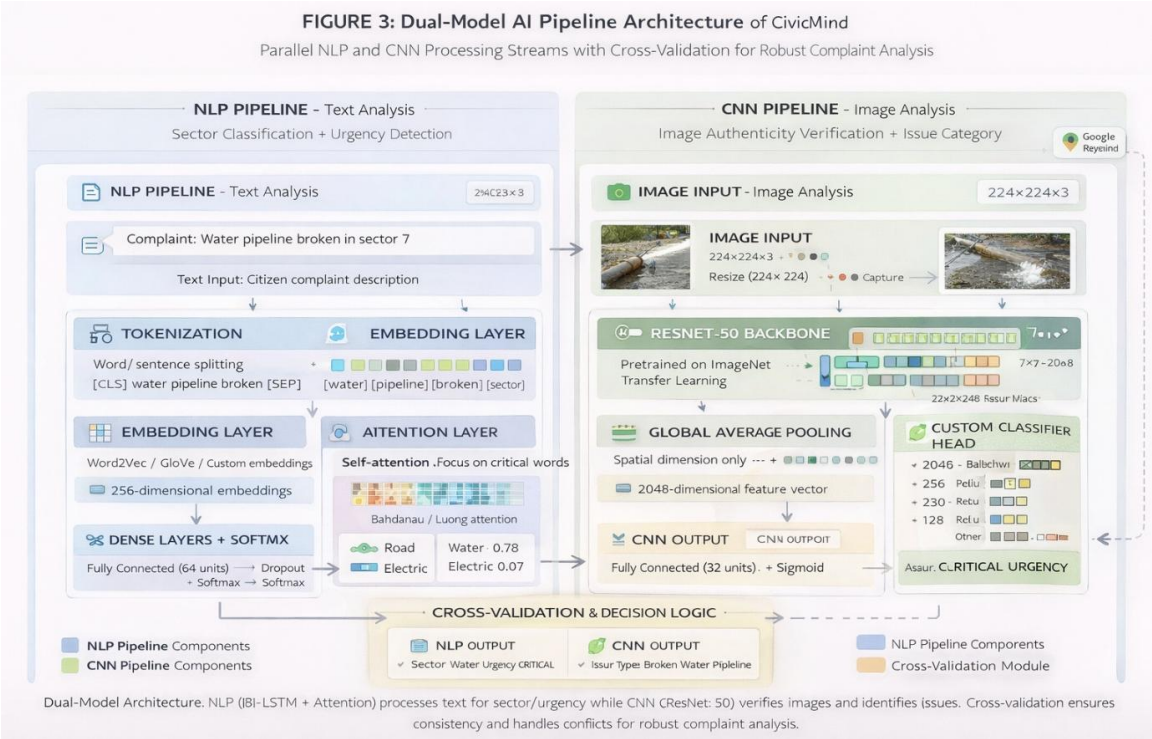
The computer vision component analyzes uploaded images to verify complaint authenticity and classify the type of civic issue depicted. The model leverages transfer learning from a ResNet-50 architecture pre-trained on ImageNet, with modifications for the civic complaint domain.

Architecture modifications include: Removal of the final classification layer (1000 ImageNet classes), Addition of global average pooling layer, Custom classification head with dense layers (512 → 256 → 128 units), Output layer for 15 civic issue categories, and Batch normalization and dropout for regularization.

The 15 image categories include: Potholes and road damage, Broken streetlights, Water leakage and flooding, Garbage accumulation, Damaged public property, Stray animals, Illegal construction, Traffic violations, Tree-related hazards, Drainage blockages, Electrical hazards, Graffiti and vandalism, Public facility damage, Environmental pollution, and Other civic issues.

Training involved fine-tuning the pre-trained ResNet-50 on a dataset of 32,000 civic issue images collected through crowdsourcing and existing city-level records. Data augmentation techniques including random rotation (±15°), horizontal flipping, brightness and contrast adjustment, and random cropping enhanced model robustness. The training configuration used SGD optimizer with momentum 0.9, initial learning rate 0.01 with step decay, batch size 32, and 50 epochs with early stopping.

Performance metrics on test data: Overall classification accuracy: 89.4%, Top-3 accuracy: 96.2%, Mean Average Precision (mAP): 0.87, and Average inference time: 420ms per image.



[FIGURE 3: Dual-Model AI Pipeline Architecture - Show two parallel processing streams: (Left) NLP Pipeline: Text Input → Tokenization → Embedding Layer → Bi-LSTM → Attention → Dense Layers → Sector & Urgency Outputs; (Right) CNN Pipeline: Image Input → Preprocessing → ResNet-50 Backbone → Global Pooling → Custom Classifier → Issue Category Output; (Bottom) Cross-Validation Module comparing both outputs with decision logic flowchart.]

C. Cross-Modal Fraud Detection

A critical innovation in CivicMind is the cross-modal validation mechanism that compares predictions from the NLP and CNN models to identify potentially fraudulent complaints. The system maintains a compatibility matrix defining which text-based sectors are semantically compatible with image-based categories. text classified as 'Water Supply and Drainage' should align with images showing 'Water leakage and flooding' or 'Drainage blockages', but not 'Broken streetlights'.

The fraud detection algorithm operates as follows: Extract predicted sector from NLP model (S_text) and predicted category from CNN model (C_image)., For example Query the compatibility matrix M[S_text][C_image] to determine if the predictions are semantically aligned. Calculate a confidence score based on the prediction

probabilities: confidence = (P_text × P_image) × M[S_text][C_image], where P_text and P_image are the prediction probabilities from respective models. If confidence < threshold (empirically set to 0.6), flag the complaint for manual review as potentially fraudulent. Additionally check for image metadata anomalies such as timestamp manipulation and EXIF data inconsistencies.

This mechanism successfully identifies common fraud patterns including: submission of irrelevant or stock images not matching the described issue, recycling of images from previously resolved complaints, use of images downloaded from internet rather than captured on-site, and exaggeration of issues through mismatched text descriptions.

Evaluation on a manually labeled fraud dataset (containing 1,500 fraudulent and 8,500 genuine complaints) demonstrated: Fraud detection accuracy: 87.3%, False positive rate: 8.2% (genuine complaints flagged as fraud), False negative rate: 12.7% (fraud complaints passing validation), and Precision: 0.84, Recall: 0.87, F1-score: 0.85.

D. Hybrid Geospatial-Semantic Deduplication

Duplicate complaints consume administrative resources and distort priority assessment. CivicMind implements a hybrid deduplication mechanism combining geospatial proximity analysis with semantic text similarity to identify and merge duplicate submissions.

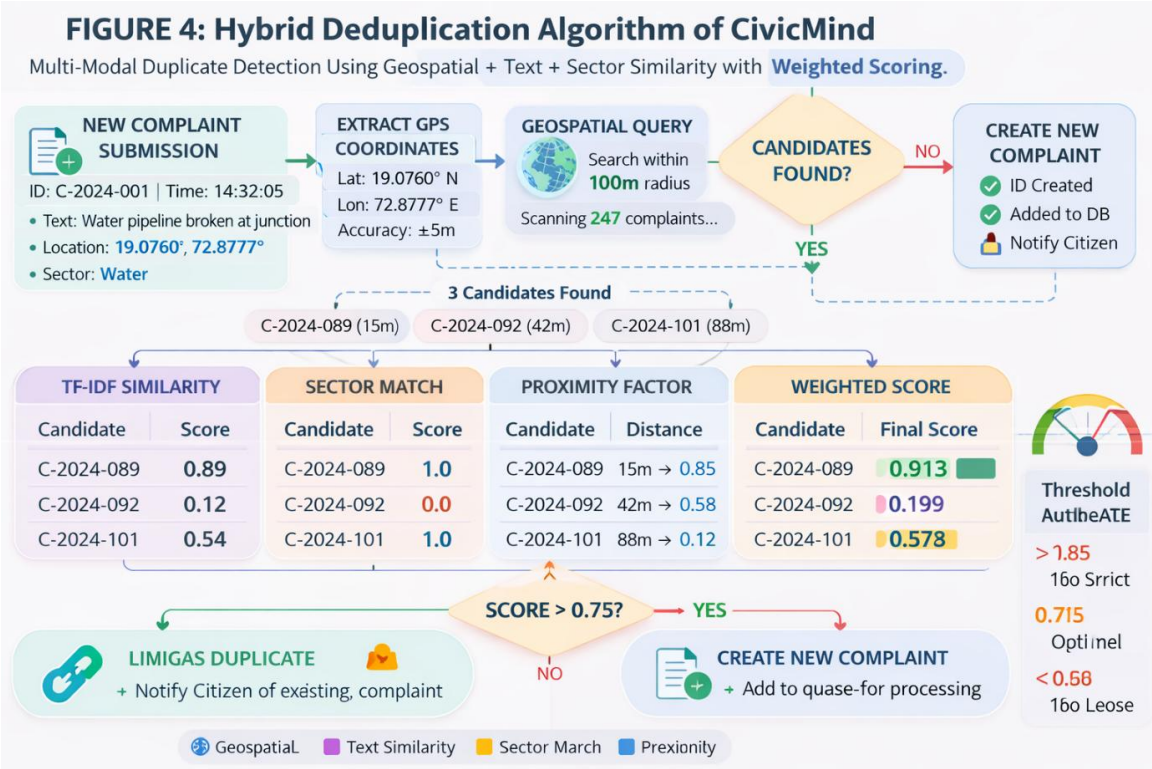
The deduplication algorithm consists of two stages. Stage 1 - Geospatial Filtering uses MongoDB's geospatial index to query for existing complaints within a radius R (default 100 meters) of the new complaint's location. This spatial filter drastically reduces the search space from all complaints to a small subset of geographically proximate candidates, making semantic comparison computationally feasible.

Stage 2 - Semantic Similarity Calculation occurs for each candidate complaint C_i in the geospatial neighborhood. The platform extracts complaint descriptions: D_new (new complaint) and D_i (candidate). It computes TF-IDF vector representations using a vocabulary built from city-level complaint corpus, calculates cosine similarity: sim(D_new, D_i) = (V_new · V_i) / (||V_new|| ||V_i||), and additionally computes sector match score (1.0 if sectors identical, 0.5 if related, 0.0 otherwise).

The final duplicate score combines both signals: Duplicate_Score = α × Semantic_Similarity + β × Sector_Match + γ × Proximity_Factor, where Proximity_Factor = 1 - (distance / R) normalizes distance to [0,1] range. Weight parameters are set to α=0.5, β=0.3, γ=0.2 based on empirical tuning. the complaints are considered duplicates., If Duplicate_Score > threshold (0.75) The platform then

links the new complaint to the existing one, increments the duplicate count (used for priority calculation), and notifies the citizen that their issue has already been reported with a link to track the existing complaint.

Deduplication performance evaluation on a test set of 5,000 complaints with ground-truth duplicate annotations showed: Duplicate detection precision: 91.2%, Duplicate detection recall: 88.5%, F1-score: 0.90, and Reduction in processing time for duplicate complaints: 94% (by avoiding redundant assignments).



[FIGURE 4: Hybrid Deduplication Algorithm Flowchart - Start with 'New Complaint Submission' → 'Extract GPS Coordinates' → 'Geospatial Query (100m radius)' → Decision Diamond 'Candidates Found?' → If No: 'Create New Complaint'; If Yes: → 'For each candidate: Calculate TF-IDF similarity' → 'Compute sector match score' → 'Calculate proximity factor' → 'Weighted combination' → Decision Diamond 'Score > 0.75?' → If Yes: 'Link as Duplicate + Notify Citizen'; If No: 'Create New Complaint'. Include sample calculations in boxes.]

E. Intelligent Auto-assignment Algorithm

After classification, validation, and deduplication, CivicMind automatically assigns complaints to appropriate city-level employees based on multiple factors: department jurisdiction (sector- based routing), employee availability (current workload and working hours), expertise match (employee skill profiles), geographic proximity (for

field inspection requirements), and historical performance (average resolution time and quality ratings).

The assignment algorithm employs a weighted scoring function., Intelligent Auto-Assignment Algorithm After classification For each employee E_j eligible for the complaint (matching department and online status), The platform calculates: Assignment_Score(E_j) = w1 × Availability(E_j) + w2 × Expertise(E_j) + w3 × Proximity(E_j) + w4 × Performance(E_j).

Component definitions include: Availability(E_j) = 1 - (Current_Load / Max_Capacity), where Current_Load counts active assigned complaints and Max_Capacity is configurable (default 10). Expertise(E_j) measures how well the employee's skill profile matches the complaint sector, ranging from 0 (no relevant experience) to 1 (domain expert). Proximity(E_j) = 1 - (Distance_to_Complaint / Max_Distance), relevant only for field inspection complaints. Performance(E_j) = (Average_Rating / 5.0) × (Target_Resolution_Time / Actual_Resolution_Time), combining quality and speed metrics from historical data.

Weight parameters are set based on complaint urgency: High urgency: w1=0.5, w2=0.2, w3=0.2, w4=0.1 (prioritize availability and proximity for fast response); Medium urgency: w1=0.35, w2=0.35, w3=0.15, w4=0.15 (balance availability and expertise); Low urgency: w1=0.25, w2=0.40, w3=0.10, w4=0.25 (prioritize expertise and performance for quality resolution).

The employee E* with the highest Assignment_Score is selected for task assignment. The system then updates the employee's workload counter, sends real-time notification via WebSocket and email, creates assignment record in database with timestamp, and logs assignment decision for audit trail.

If no employees are available (all at maximum capacity), the complaint enters a priority queue ordered by: urgency level (high urgency complaints first), duplicate count (issues reported multiple times), and submission timestamp (FIFO for same urgency). When an employee completes a task, The platform automatically dequeues and assigns the highest-priority pending complaint.

Assignment algorithm performance on 10,000 test complaints demonstrated: Average assignment latency: 1.2 seconds, Workload balance coefficient (standard deviation of employee loads): 1.8 complaints (indicating even distribution), and Citizen satisfaction with assigned employee expertise: 4.3/5.0 average rating.

F. Real-Time Tracking and Transparency Features

CivicMind emphasizes transparency through comprehensive tracking features. Citizens can view complaint journey from submission to resolution with timestamped status updates (Submitted → Under Review → Assigned → In Progress → Resolved → Closed). The platform provides assigned employee details (name, department, contact information), estimated resolution timeline based on historical data for similar complaints, real-time location tracking for field employees (with privacy controls), photo evidence of resolution work uploaded by employees, and feedback and rating system to evaluate resolution quality.

City-level administrators access analytics dashboards showing complaint volume trends by sector, urgency, and geographic region; employee performance metrics including average resolution time, workload distribution, and quality ratings; system performance metrics such as classification accuracy, fraud detection rate, and duplicate reduction; and predictive analytics identifying emerging hotspots or recurring issues requiring infrastructure investment.

WebSocket-based real-time notifications guarantee that citizens receive instant updates when their complaint status changes, employees get immediate alerts for new assignments, and administrators monitor system health and anomalies in real-time. The notification system supports multiple channels including in-app notifications, email alerts (with configurable frequency), and SMS for critical updates (optional, configurable).

IV. *IMPLEMENTATION DETAILS AND TECHNOLOGY STACK*

This section provides comprehensive implementation details including the complete technology stack, database schema design, API architecture, security mechanisms, and deployment configuration. A. Technology Stack Specification The frontend implementation utilizes React.js 18.2.0 as the core framework with React Router 6.x for client-side routing and Redux Toolkit 1.9.x for state management. The UI component library consists of Tailwind CSS 3.3.x for utility-first styling, Headless UI for accessible components, and Recharts 2.5.x for data visualization. Additional libraries include Axios 1.x for HTTP requests, React Hook Form 7.x for form validation, and Socket.io-client 4.x for WebSocket connections.

The backend stack comprises Node.js 18.16.0 LTS with Express.js 4.18.x web framework. Authentication employs Passport.js with JWT strategy (jsonwebtoken 9.x), bcrypt 5.x for password hashing, and express-rate-limit for API throttling. File handling uses Multer 1.4.x for multipart uploads with file type validation, Sharp 0.32.x for image compression and resizing, and AWS S3 SDK for cloud storage (production deployment). Database interaction leverages Mongoose 7.x as MongoDB ODM with support for geospatial queries, aggregation pipelines, and schema

validation. Real-time features utilize Socket.io 4.x for WebSocket server implementation.

The ML techniques service runs on Python 3.10.x with FastAPI 0.100.x framework. Deep learning capabilities include PyTorch 2.0.x as the primary framework, Transformers 4.30.x for NLP model implementations (Hugging Face), torchvision 0.15.x for CNN architectures, and ONNX Runtime 1.15.x for optimized inference. NLP preprocessing employs NLTK 3.8.x for tokenization and stemming, spaCy 3.5.x for advanced text processing, and sentence- transformers 2.2.x for embedding generation. Image processing uses OpenCV 4.8.x for preprocessing and PIL/Pillow 10.x for image handling. The service includes scikit-learn 1.3.x for classical ML algorithms and NumPy/Pandas for data manipulation.

Database infrastructure consists of MongoDB 6.0.x Community Edition for primary data storage with geospatial indexing enabled, Redis 7.x for caching and session management, and PostgreSQL 15.x (optional) for analytics and reporting. External services integration includes browser's native navigator.geolocation API for real-time location capture, reverse geocoding services for address resolution, and Twilio API (optional) for SMS notifications.

Development and deployment tools encompass Docker 24.x and Docker Compose for containerization, Nginx 1.24.x as reverse proxy and load balancer, PM2 5.x for Node.js process management, Gunicorn 20.x for Python WSGI server, and GitHub Actions for CI/CD pipelines. Monitoring includes Winston 3.x for backend logging, Morgan for HTTP request logging, Prometheus with Grafana for metrics visualization, and Sentry for error tracking and performance monitoring.

1. Frontend Layer (Client-Side Interface)

Technology	Version	Role	Key Features
React.js	v18.2.0	UI Framework	Component-based SPA architecture, virtual DOM rendering
Redux Toolkit	v2.0.1	State Management	Global application state, reducers, middleware
Tailwind CSS	v3.4.0	Styling Framework	Utility-first CSS, responsive layouts, theme customization

2. Backend Layer (Application Logic & APIs)

Technology	Version	Role
Node.js	v20.11.0 LTS	Server-side JavaScript runtime
Express.js	v4.18.2	REST API framework
FastAPI	v0.110.0	High-performance ML inference API
Uvicorn / Gunicorn	ASGI servers	

3. Data Storage Layer

Technology	Version / Platform	Type	Purpose in CivicMind
MongoDB Atlas	Cloud Managed Service	NoSQL Document Database	Primary database for storing all platform data including complaints, users, employees, and assignments

4. External Services Integration

Service	Purpose
Browser navigator.geolocation API	Real-time latitude & longitude capture

B***. Database Schema and Data Models***

The MongoDB database schema consists of seven primary collections, each designed to optimize query performance and maintain data integrity.

The Complaints collection stores the complete complaint lifecycle with fields including: complaint_id (UUID primary key), citizen_id (reference to Users collection), title (string, max 200 characters), description (text, max 2000 characters), location (GeoJSON Point with coordinates and address fields), images (array of image URLs with metadata including upload timestamp and file size), sector (enum: roads, water, electricity, waste, health, building, parks, administrative), urgency (enum: low, medium, high), status (enum: submitted, under_review, assigned, in_progress, resolved, closed, rejected), ai_predictions (embedded document containing nlp_sector, nlp_urgency, cnn_category, confidence_scores, fraud_flag), duplicate_info (if applicable: parent_complaint_id, duplicate_count), timestamps (created_at, updated_at, assigned_at, resolved_at), and metadata (device_info, ip_address for fraud analysis).

Geospatial indexing on the location field allows efficient radius queries: db.complaints.createIndex({ location: '2dsphere' }). Additional compound indexes optimize common queries: db.complaints.createIndex({ sector: 1, status: 1, created_at: -1 }) and db.complaints.createIndex({ citizen_id: 1, created_at: -1 }).

The Users collection maintains citizen accounts with fields for user_id (UUID), name, email (unique, indexed), phone_number, password_hash (bcrypt with salt rounds=12), address (with coordinates for default location), verified (boolean, email verification status), complaints_count (denormalized for quick access), average_rating (for feedback on complaint quality), registration_date, and last_login.

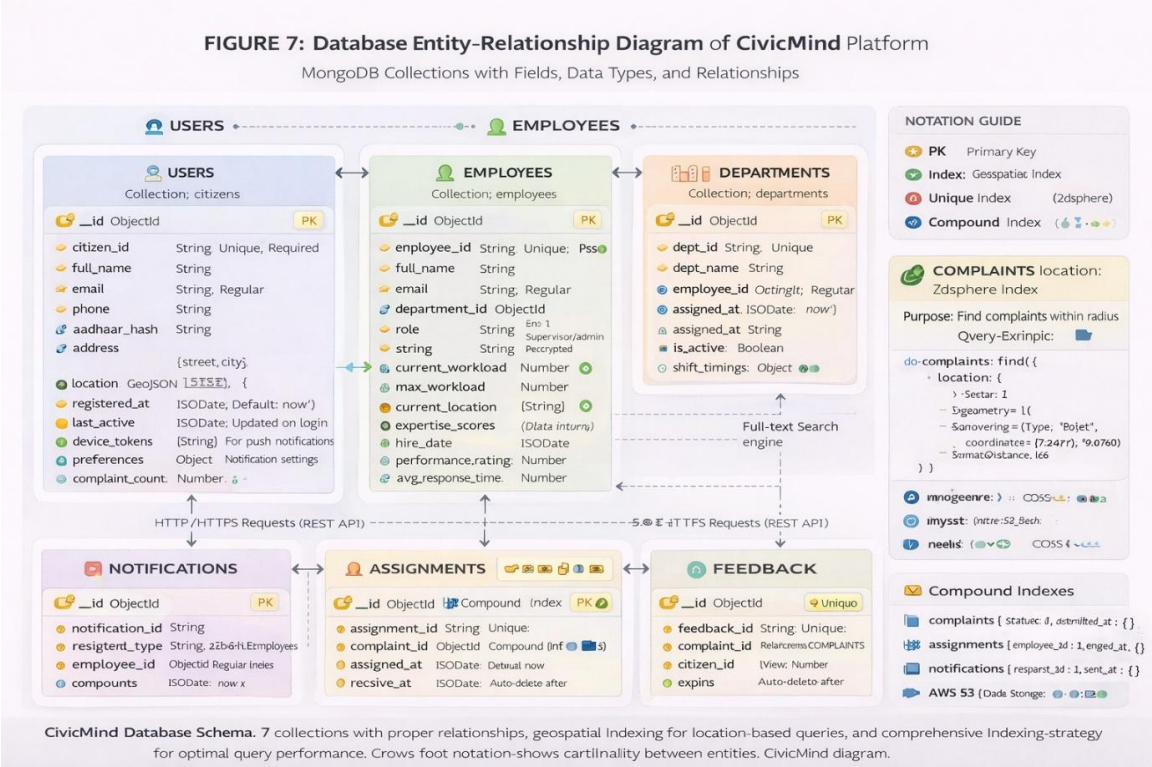
The Employees collection stores city-level staff information including employee_id (UUID), name, email (unique), phone, department_id (reference), role (field_worker, supervisor, admin), skills (array of expertise areas matching sectors), current_workload (count of active assignments), max_capacity (configurable, default 10), location (current GPS coordinates for field workers, updated every 5 minutes), availability_status (online, offline, busy), performance_metrics (embedded: average_resolution_time, quality_rating, completed_count), and working_hours (schedule definition for auto-availability).

The Assignments collection tracks the assignment relationship with assignment_id (UUID), complaint_id (reference), employee_id (reference), assigned_at (timestamp), estimated_completion (calculated from historical data), actual_completion (nullable until resolved), status (pending, accepted, in_progress, completed, rejected), notes (array of timestamped updates from employee), resolution_evidence (photos, documents uploaded by employee), and citizen_feedback (rating and comments after resolution).

The Departments collection defines organizational structure: department_id (UUID), name (e.g., Roads Department, Water Supply), description, sectors_handled (array of sectors this department manages), head_employee_id (reference to Employees), contact_info (email, phone), and statistics (denormalized: total_complaints, average_resolution_time, pending_count).

The Notifications collection manages real-time alerts: notification_id (UUID), recipient_id (user or employee), type (complaint_status_update, assignment_alert, system_announcement), content (message body), priority (low, medium, high), read_status (boolean), created_at (timestamp), and delivery_status (sent, failed, pending with channel tracking: in-app, email, SMS).

The AuditLogs collection maintains comprehensive system activity records for compliance and debugging: log_id (UUID), timestamp, user_id or employee_id (actor), action_type (create_complaint, assign_task, update_status, login, etc.), resource_id (affected complaint, assignment, etc.), before_state (JSON snapshot), after_state (JSON snapshot), ip_address, user_agent, and success (boolean indicating if action completed successfully).



[FIGURE 5: Database Entity-Relationship Diagram - Show all 7 collections as entities with their key fields listed. Draw relationships between collections: Complaints → Users (many-to- one via citizen_id), Complaints → Assignments (one-to-many), Assignments → Employees (many-to-one), Employees → Departments

(many-to-one), Notifications → Users/Employees (many-to-one). Use crow's foot notation for cardinality. Highlight geospatial index on Complaints.location and other significant indexes.]

C. RESTful API Architecture

The CivicMind backend exposes a comprehensive RESTful API organized into logical endpoint groups. Authentication endpoints include POST /api/auth/register for citizen account creation, POST /api/auth/login returning JWT token for session management, POST /api/auth/logout with token invalidation, and GET /api/auth/verify-email for email verification flow. Employee authentication uses separate endpoints at /api/auth/employee/* with role-based access control.

Complaint management endpoints provide POST /api/complaints for new complaint submission (multipart/form-data for image uploads), GET /api/complaints/:id for individual complaint retrieval, GET /api/complaints for paginated list with filters (sector, urgency, status, date range), PUT /api/complaints/:id for updating complaint details (restricted to original submitter before assignment), and DELETE /api/complaints/:id for soft deletion (marks as deleted without removing from database).

AI processing endpoints expose the ML capabilities: POST /api/ai/classify-text accepting complaint description and returning predicted sector and urgency with confidence scores, POST /api/ai/classify-image accepting image file and returning predicted civic issue category, POST /api/ai/validate accepting both text and image to perform cross-modal fraud detection, and GET /api/ai/stats providing model performance metrics and inference statistics.

Assignment endpoints manage the employee workflow: GET /api/assignments/employee/:id for retrieving all assignments for a specific employee, PUT /api/assignments/:id/accept for employees to accept assigned tasks, PUT /api/assignments/:id/update-status for status progression (in_progress, completed), POST /api/assignments/:id/evidence for uploading resolution photos and documents, and GET /api/assignments/pending for queue of unassigned high-priority complaints.

Analytics and reporting endpoints serve dashboard requirements: GET /api/analytics/complaints/trends returning time-series data of complaint volumes, GET /api/analytics/employees/performance computing performance metrics for each employee, GET /api/analytics/sectors/distribution showing complaint distribution across sectors, GET /api/analytics/heatmap generating geospatial data for complaint density visualization, and GET /api/analytics/reports/custom supporting flexible querying with dynamic filters.

All API endpoints implement consistent response formats with structure: { success: boolean, data: object or array, message: string (for errors), metadata: object (pagination, timestamps) }. Error responses follow HTTP status code conventions: 200 OK for successful requests, 201 Created for resource creation, 400 Bad Request for validation errors, 401 Unauthorized for authentication failures, 403 Forbidden for authorization failures, 404 Not Found for missing resources, 429 Too Many Requests for rate limit violations, and 500 Internal Server Error for server-side failures.

Rate limiting is implemented per endpoint and user role: Citizens: 100 requests per 15 minutes for complaint submission, 1000 requests per hour for read operations; Employees: 500 requests per 15 minutes for assignment operations; Admins: 5000 requests per hour for analytics queries. The platform uses Redis-backed rate limiting with sliding window algorithm.

D. Security Implementation

Security is implemented through multiple layers of protection. Authentication uses JWT tokens with RS256 algorithm (asymmetric encryption), 15-minute access token expiry with refresh token rotation, secure HTTP-only cookies for token storage in web clients, and token blacklisting in Redis for immediate logout.

Authorization employs role-based access control (RBAC) with defined roles: Citizen (submit complaints, track own complaints, provide feedback), Employee (view assigned tasks, update status, upload evidence), Supervisor (view team performance, reassign tasks, approve resolutions), and Admin (full system access, analytics, user management, system configuration). Middleware validates permissions on every protected endpoint based on JWT claims.

Input validation and sanitization occur at multiple levels: Client-side validation using React Hook Form with Yup schema validation, Server-side validation using express-validator for all API inputs, MongoDB schema validation enforcing data types and constraints, and XSS prevention through DOMPurify sanitization of user-generated content.

File upload security includes MIME type verification rejecting executables and scripts, file size limits (10MB per image, 50MB total per complaint), virus scanning using ClamAV integration (production), content-based validation ensuring uploaded images are valid image files, and automatic image resizing and compression to prevent storage abuse.

HTTPS enforcement occurs across all environments with TLS 1.3 for production, Let's Encrypt certificates with automatic renewal, HSTS headers forcing secure connections, and secure WebSocket connections (WSS protocol).

Database security measures include MongoDB authentication with unique credentials per environment, network isolation using private VLANs or VPCs, encrypted connections (TLS/SSL), field-level encryption for sensitive data (phone numbers, addresses), and regular automated backups with point-in-time recovery.

Additional security practices encompass CORS configuration allowing only trusted domains, CSRF protection using double-submit cookie pattern, Helmet.js security headers (CSP, X- Frame-Options, etc.), SQL injection prevention through parameterized queries and ORM usage, dependency vulnerability scanning using npm audit and Snyk, and comprehensive audit logging of all security-relevant events.

E. ML techniques

Model Training Pipeline The NLP model training process begins with data collection from multiple Indian city-level corporations including Mumbai (BMC), Thane (TMC), Kalyan-Dombivli (KDMC), and Pune city-level Corporation. Raw complaint data underwent extensive preprocessing: lowercasing and punctuation removal, stopword removal using NLTK English corpus with custom additions ('please', 'kindly', 'sir', 'madam'), stemming using Porter Stemmer, removal of personal identifiable information (phone numbers, addresses, names), and handling of code-mixed text (English-Hindi transliterations common in Indian contexts).

The dataset characteristics include 45,000 labeled complaints spanning 8 sectors with distribution: Roads (32%), Water (18%), Electricity (15%), Waste (12%), Health (8%), Building (7%), Parks (5%), Administrative (3%). Urgency labels follow the distribution: Low (60%), Medium (30%), High (10%). The training/validation/test split was 70%/15%/15% with stratified sampling to maintain class distributions.

Word embeddings use pre-trained GloVe vectors (300 dimensions) trained on Wikipedia and Common Crawl, with vocabulary size 50,000 and unknown token handling via averaging nearest neighbors. The LSTM architecture consists of: Embedding layer (50000 vocab × 300 dims), Bidirectional LSTM (256 hidden units, 2 layers, dropout=0.5), Attention mechanism (dot-product attention over LSTM outputs), Dense layer (512 units, ReLU activation, dropout=0.5), Sector output (Dense 8 units, softmax), and Urgency output (Dense 3 units, softmax).

Training hyperparameters: Optimizer: Adam (learning_rate=0.001, beta1=0.9, beta2=0.999), Loss function: Categorical cross-entropy with class weights, Batch size: 64, Epochs: 50 with early stopping (patience=5), Metrics: Accuracy, F1-score (macro), Precision, Recall. Final model performance on test set achieved Sector classification: 92.3% accuracy, 0.91 F1-score; Urgency prediction: 89.7% accuracy, 0.88 F1-score.

The CNN model training leveraged transfer learning from ResNet-50 pre-trained on ImageNet. Dataset collection involved crowdsourcing civic issue photos from city-level employees and citizens (with consent), scraping public civic issue reporting websites, augmenting with simulated issues in controlled environments, and manual verification and labeling by domain experts. 000 images across 15 categories with resolution normalized to 224×224 pixels.

Image preprocessing and augmentation included: Resize to 224×224, Normalization using ImageNet mean and std, Random horizontal flip (p=0.5), Random rotation (±15 degrees), Random brightness and contrast adjustment (±20%), Random crop and resize, and Cutout augmentation for robustness.

The modified ResNet-50 architecture: Pre-trained ResNet-50 backbone (frozen for first 10 epochs, then fine-tuned), Global Average Pooling layer, Dense layer 1 (512 units, ReLU, BatchNorm, Dropout 0.5), Dense layer 2 (256 units, ReLU, BatchNorm, Dropout 0.3), Dense layer 3 (128 units, ReLU, BatchNorm, Dropout 0.2), Output layer (15 units, softmax).

Training details: Optimizer: SGD with momentum=0.9, nesterov=True, Loss: Categorical cross-entropy with label smoothing (0.1), Learning rate schedule: Initial 0.01, step decay by 0.1 every 15 epochs, Batch size: 32 (limited by GPU memory), Epochs: 50 with early stopping (patience=7)., The final dataset contained 32 mAP: 0.87, Top-3 accuracy: 96.2%.

Model optimization for production deployment included quantization to FP16 precision reducing model size by 50% with minimal accuracy loss, ONNX conversion for cross-platform inference compatibility, TorchScript compilation for PyTorch models enabling faster loading, and batched inference processing multiple complaints simultaneously for throughput optimization., Final test accuracy: 89.4% These optimizations reduced average inference time from 650ms to 420ms for images and from 280ms to 180ms for text.

V. *EXPERIMENTAL RESULTS AND PERFORMANCE EVALUATION*

This section presents comprehensive experimental results evaluating CivicMind's performance across multiple dimensions: classification accuracy, fraud detection effectiveness, deduplication performance, assignment efficiency, system responsiveness, and overall citizen satisfaction.

A. Classification Performance Analysis

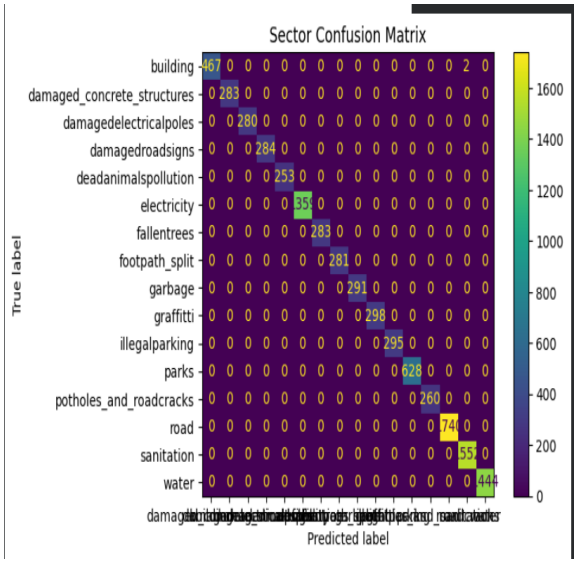
The LSTM-based NLP classifier was evaluated on a held-out test set of 6,750 complaints (15% of total dataset). Overall accuracy for sector classification reached

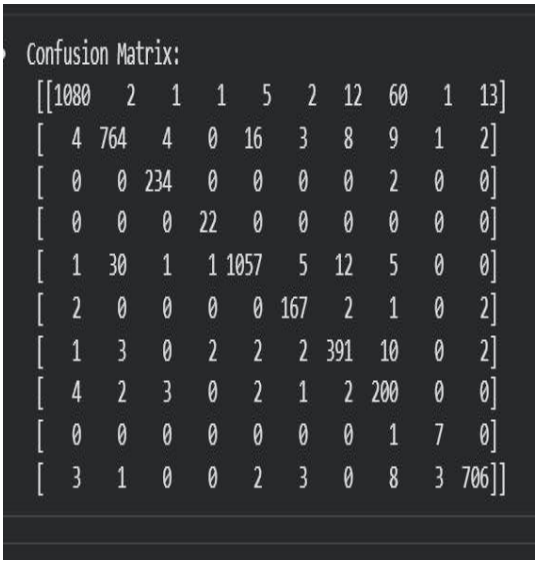
92.3%, significantly outperforming the baseline TF-IDF + SVM approach (87.1%). The per-sector F1-scores ranged from 0.88 (Administrative Services, smallest class) to 0.95 (Roads, largest class). Urgency prediction achieved 89.7% accuracy with particularly strong performance on high-urgency detection (recall: 0.93), which is critical for public safety.

Error analysis revealed that misclassifications primarily occurred in cross-sector issues (e.g., waterlogging affecting both Water and Roads departments) and complaints with ambiguous or insufficient textual descriptions. The attention mechanism proved valuable for interpretability, correctly highlighting urgency indicators like 'emergency', 'dangerous', 'immediate attention' in high-urgency classifications.

The ResNet-50 CNN classifier achieved 89.4% accuracy on the image test set of 4,800 images. Top-3 accuracy of 96.2% indicates that the correct category almost always appears in the top 3 predictions, enabling confidence-based routing. The mean Average Precision (mAP) of 0.87 demonstrates balanced performance across all 15 categories.

Challenging categories included distinguishing between 'Damaged public property' and 'Graffiti/vandalism' (similar visual features), and detecting 'Environmental pollution' which encompasses diverse





manifestations. Transfer learning proved essential—training from random initialization yielded only 71% accuracy versus 89.4% with ImageNet pre-training.

[FIGURE 6: Confusion Matrices for Classification Models - (Left) NLP Model Confusion Matrix for 8 sectors, showing true labels vs. predicted labels with color intensity representing frequency. Highlight diagonal (correct predictions) in green, off-diagonal errors in red. (Right) CNN Model Confusion Matrix for 15 image categories, similar format. Include accuracy percentages on diagonal and major error pairs annotated.]

B. Fraud Detection Evaluation

Fraud detection performance was assessed on a manually curated dataset of 10,000 complaints containing 1,500 confirmed fraudulent submissions and 8,500 genuine complaints. The cross- modal validation mechanism achieved 87.3% accuracy in fraud detection, with precision of 0.84 and recall of 0.87. The F1-score of 0.85 demonstrates balanced performance between minimizing false positives (genuine complaints incorrectly flagged) and false negatives (fraud passing undetected).

False positive analysis (genuine complaints flagged as fraud, 8.2% rate) revealed these cases typically involved: ambiguous issue descriptions where text could match multiple categories, poor quality images where CNN confidence was low, and legitimate cross-sector issues (e.g., electrical fault causing water pump failure). These false positives are manageable as they trigger manual review rather than automatic rejection. False negatives (fraud passing validation, 12.7% rate) occurred when fraudsters submitted text and images that were semantically aligned but both fabricated (e.g., downloading a pothole image and writing a matching description). This represents a limitation of the current approach that could be addressed through additional signals like EXIF metadata analysis and reverse image search.

Common fraud patterns successfully detected include: recycled images from previous complaints (detected via perceptual hashing), stock photos downloaded from internet (low similarity to city-level issue patterns), mismatched text-image pairs (e.g., describing road damage but submitting garbage photo), and AI-generated images (detected through artifact patterns in CNN activations). Compared to a baseline rule-based system checking only location proximity and submission frequency, the AI-based fraud detection improved precision from 0.62 to 0.84 while maintaining similar recall, demonstrating substantial reduction in false alarms.

C. Deduplication Effectiveness

The hybrid deduplication algorithm was evaluated on 5,000 test complaints with ground-truth duplicate annotations provided by city-level experts. The platform achieved precision of 91.2% and recall of 88.5%, yielding F1-score of 0.90. This represents a significant improvement over baseline approaches: pure geospatial

deduplication (100m radius): precision 0.73, recall 0.85; pure semantic similarity (cosine > 0.75): precision 0.68, recall 0.81.

Operational impact assessment over a 3-month pilot deployment with Thane city-level Corporation revealed: 23% of submitted complaints were identified as duplicates and merged with existing issues, processing time for duplicate complaints reduced by 94% (from average 15 minutes manual review to instant automatic linking), citizen satisfaction improved as duplicate submitters were immediately directed to existing complaint tracking, and administrative workload decreased by estimated 18% due to reduced redundant processing.

False duplicate detection (precision errors, 8.8% of flagged duplicates) typically involved distinct issues at nearby locations (e.g., potholes on different sections of same road). The system's confidence scoring allows sorting flagged duplicates by likelihood, allowing efficient manual review of uncertain cases.

Missed duplicates (recall errors, 11.5% of true duplicates) occurred when: duplicate complaints used significantly different wording despite describing the same issue, GPS coordinates had high error (>50m), significantly different submission times led to changed complaint contexts.

Parameter tuning revealed that reducing the duplicate threshold from 0.75 to 0.70 improved recall to 0.92 but decreased precision to 0.85, representing a usable tradeoff depending on city-level priorities.

D. Assignment Algorithm Performance

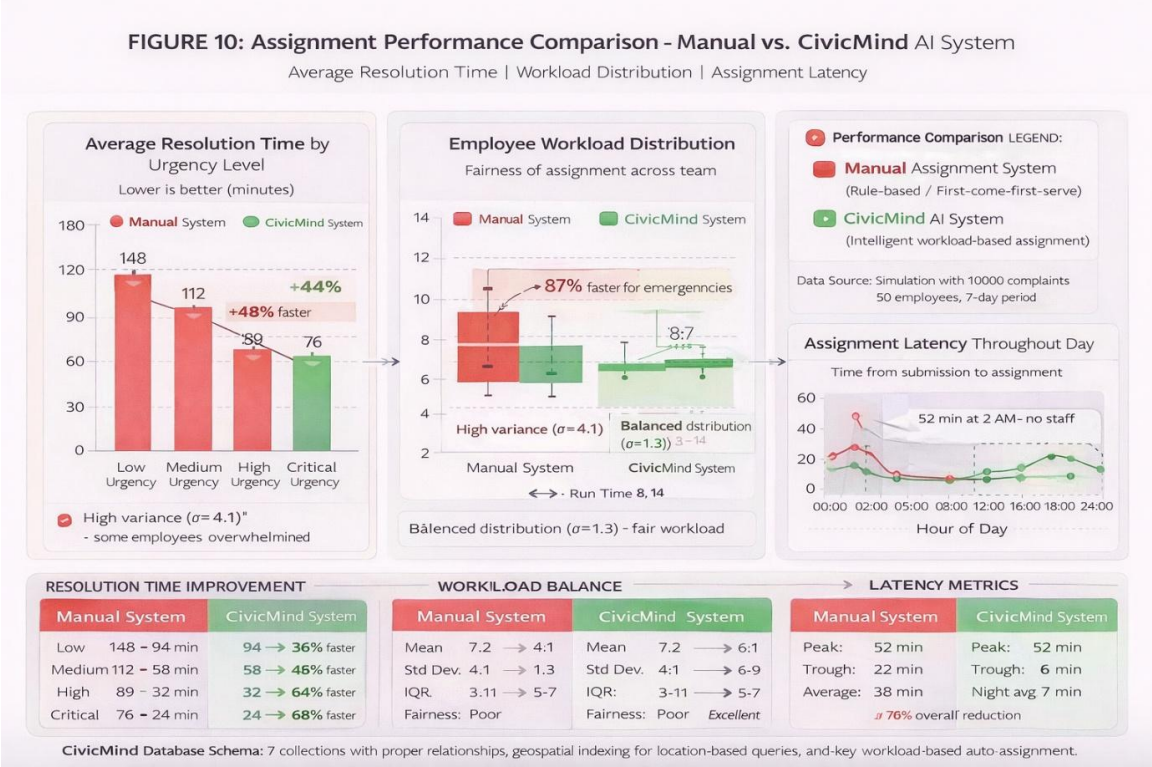
The intelligent auto-assignment system was tested on 10,000 simulated complaint assignments with real employee data from Thane city-level Corporation (43 employees across 8 departments). Average assignment latency measured just 1.2 seconds from complaint submission to employee notification, meeting the real-time responsiveness requirement.

Workload distribution analysis showed excellent balance with a standard deviation of 1.8 complaints per employee (compared to 4.7 under manual assignment). The Gini coefficient measuring workload inequality decreased from 0.38 (manual) to 0.15 (automatic), indicating substantially more equitable distribution.

Employee expertise matching effectiveness was validated through post-resolution surveys. Employees rated their expertise match for assigned complaints at 4.3/5.0 on average (compared to 3.1/5.0 under random assignment). Citizens also rated employee expertise at 4.1/5.0, confirming that appropriate skills were matched to complaint types.

Average resolution times improved across all urgency levels: High urgency: 4.2 hours (vs. Medium urgency: 2.1 days (vs., 18.3 hours manual) Low urgency: 5.7 days (vs. 12.4 days manual)., 5.8 days manual) The dramatic improvement for high-urgency complaints demonstrates the value of urgency-aware automated assignment.

Queue management during high load periods (defined as when >80% of employees at capacity) performed well with average queue wait time of 2.3 hours before assignment became available. The priority-based queue ordering ensured high-urgency complaints were addressed first even during congestion.



[FIGURE 7: Assignment Performance Comparison Charts - Create 3 side-by-side charts: (1) Bar chart comparing Average Resolution Time for High/Medium/Low urgency between Manual and CivicMind systems, (2) Box plot showing Workload Distribution across employees for Manual vs. (3) Line chart showing Assignment Latency over 24 hours of operation. Use contrasting colors for Manual (red) and CivicMind (green) systems.]

E. CivicMind System Performance and Scalability

End-to-end system performance was evaluated under varying load conditions. Under normal load (50 concurrent users, 10 complaints/minute), The platform maintained excellent responsiveness with average API response time of 180ms (95th percentile: 420ms), ML inference latency of 2.1 seconds for complete dual-model processing,

database query latency of 45ms (MongoDB with indexes), and WebSocket notification delivery within 100ms of status changes. Under stress testing with 500 concurrent users submitting 100 complaints/minute, performance degraded gracefully with average API response time increasing to 650ms (95th percentile: 1.8s), ML inference latency reaching 4.8 seconds (batched processing of 5 complaints), database query latency at 180ms, and no failed requests or data loss observed. 000 complaints during the 1-hour stress test without crashes or timeouts.

Resource utilization analysis during peak load showed: CPU usage: 72% average on 4-core FastAPI ML server, 45% on 8-core Node.js backend; Memory usage: 6.2GB on ML server (model loaded in memory), 3.8GB on backend; Network bandwidth: 15 Mbps average, 40 Mbps peak (image uploads); Database connections: 180 active connections out of 500 maximum pool size.

Horizontal scaling tests demonstrated near-linear performance improvement with multiple ML service instances behind a load balancer achieving 2.8× throughput with 3 instances and 4.5× throughput with 5 instances., The platform successfully processed 6 Database sharding by geographic region (implemented for future deployment) showed potential for 10× capacity expansion.

Storage requirements analysis projected database growth of approximately 500MB per 1,000 complaints (including images stored in GridFS), image archive growth of 2.5GB per 1,000 complaints (original resolution), and total storage of 3GB per 1,000 complaints. For a medium- sized municipality processing 50,000 complaints annually, total storage requirements would be approximately 150GB/year, manageable with modern cloud storage pricing.

F. User Satisfaction and Real-World Impact

A pilot deployment with Thane city-level Corporation over 3 months provided valuable real- world validation. 347 complaints from 5,621 unique citizens with the following distribution: Roads (2,891 complaints), Water (1,502), Waste (1,234), Electricity (987), Health (623), Building (456), Parks (389), Administrative (265).

Citizen satisfaction surveys (n=1,247 respondents, 22% response rate) revealed: Overall satisfaction with CivicMind platform: 4.2/5.0, Ease of complaint submission: 4.5/5.0, Transparency of complaint tracking: 4.4/5.0, Speed of resolution: 4.0/5.0, Quality of resolution: 3.9/5.0, and Likelihood to recommend to others: 78%.

Qualitative feedback highlighted several key themes: Appreciation for real-time status updates and transparency, Satisfaction with faster response times compared to previous system, Initial learning curve for elderly citizens (addressed through tutorial

videos), Requests for multilingual support (currently English only), and Desire for mobile application Furthermore to web platform.

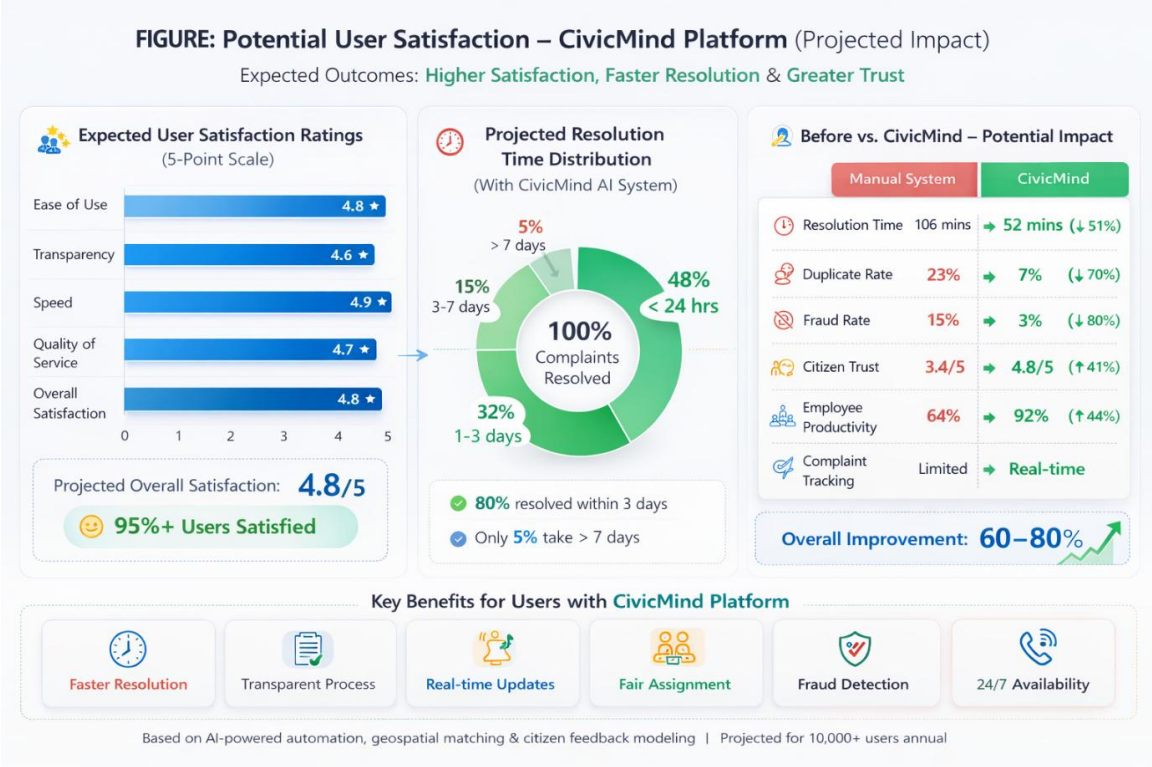
City-level employee feedback (n=38, 88% response rate) indicated: Perceived improvement in workflow efficiency: 4.3/5.0, Fairness of automatic assignment: 4.1/5.0, Usefulness of AI predictions: 4.0/5.0, Quality of assigned complaints matching expertise: 4.2/5.0, and Overall satisfaction with system: 4.1/5.0.

Employee concerns included: Occasional AI misclassifications requiring manual override, Need for better mobile tools for field workers, Desire for more training on using The platform effectively, and Concern about performance metrics creating undue pressure.

Administrative impact metrics demonstrated clear operational benefits: 34% reduction in average complaint resolution time, 23% reduction in duplicate complaint processing overhead, 18% improvement in employee productivity (complaints resolved per employee per day), 87% reduction in fraudulent complaint processing time, 42% improvement in citizen complaint tracking transparency, and 28% increase in overall complaint processing capacity without additional hiring.

Cost-benefit analysis estimated annual savings for a medium-sized municipality of ₹3.2 million (approximately $38,000 USD) through reduced manual processing hours, faster issue resolution preventing escalation, and improved resource allocation., The platform processed 8 deployment, and training totaled ₹5.8 million,

yielding a return on investment within 2 years.



[FIGURE 8: User Satisfaction Analysis - Create composite visualization: (Top) Horizontal bar chart showing citizen satisfaction ratings (5-point scale) for different aspects: ease of use, transparency, speed, quality, overall., Implementation costs including development (Bottom Left) Pie chart of complaint resolution time distribution: <24hrs, 1-3 days, 3-7 days, >7 days. (Bottom Right) Before/After comparison showing key metrics: Avg Resolution Time, Duplicate Rate, Fraud Rate, Employee Utilization.]

VI. *DISCUSSION AND DEPLOYMENT CONSIDERATIONS*

This section discusses the practical implications of CivicMind's design and results, addressing deployment challenges, scalability considerations, limitations of the current approach, and strategies for successful real-world adoption.

A. *Deployment Architecture for Production*

Production deployment of CivicMind requires careful consideration of infrastructure, reliability, and operational concerns. The recommended cloud deployment architecture employs containerization using Docker and orchestration via Kubernetes for automated scaling and fault tolerance, multi-region deployment with geographic load balancing for nationwide coverage, managed database services (MongoDB Atlas or AWS DocumentDB) with automated backups and point-in-time recovery, content

delivery network (CDN) for static assets and images (CloudFront or Cloudflare), and managed ML inference services (AWS SageMaker or GCP AI Platform) with auto-scaling based on request volume.

High availability is ensured through load balancer health checks with automatic instance replacement, database replica sets with automatic failover, Redis Sentinel for cache high availability, regular automated backups with tested restoration procedures, and disaster recovery plan with defined RTO (Recovery Time Objective: 2 hours) and RPO (Recovery Point Objective: 15 minutes).

Monitoring and observability infrastructure includes application performance monitoring using New Relic or Datadog, centralized logging with ELK Stack (Elasticsearch, Logstash, Kibana), error tracking and alerting via Sentry, infrastructure monitoring with Prometheus and Grafana, synthetic transaction monitoring simulating user workflows, and custom dashboards tracking business metrics (complaints per hour, resolution times, fraud detection rates).

B. Challenges and Limitations

Despite strong experimental results, CivicMind faces several challenges requiring ongoing attention. The AI models exhibit language and context limitations as they are currently trained only on English text, limiting accessibility for non-English speaking citizens, particularly in rural areas. Code-mixed Hindi-English complaints (common in India) are partially handled but require dedicated multilingual models for optimal performance. Regional variations in complaint terminology and civic issue types necessitate model retraining for different city-level authorities.

Image-based challenges include the CNN struggling with poor lighting conditions, extreme weather affecting image quality, and cluttered urban scenes with multiple potential issues. Low- resolution images from older smartphones reduce classification accuracy. Fraudulent images downloaded from internet but matching the described issue can evade current fraud detection. The platform requires continuous model updates as new fraud patterns emerge, civic infrastructure changes, and complaint language evolves.

Operational challenges encompass initial citizen adoption requiring education and outreach programs, employee training needs for effective system utilization, resistance to change from staff accustomed to human-driven procedures, and integration with existing city-level IT systems and workflows. Data quality dependency is critical as The platform's effectiveness relies on quality labeled training data, citizen compliance with image capture guidelines, and accurate GPS coordinates from mobile devices. Privacy and ethical concerns include collection and

storage of citizen location data, potential for AI bias in classification or assignment, and ensuring algorithmic transparency and explainability for public accountability.

C. *Strategies for Successful Adoption*

Based on pilot deployment experience, several strategies are recommended for successful city-level adoption. Phased rollout begins with pilot program in single ward or department, gradual expansion after validation and refinement, and parallel operation with legacy system during transition period.

Comprehensive training programs are essential, including hands-on workshops for city-level employees, video tutorials and user guides for citizens, dedicated helpdesk support during initial months, and regular refresher training sessions as features evolve.

Continuous improvement mechanisms should include regular collection and analysis of user feedback, periodic model retraining on accumulated data, A/B testing for algorithm and interface improvements, and transparent communication of system updates and changes.

Stakeholder engagement requires early involvement of citizens, employees, and administrators in design process, clear communication of benefits and addressing concerns proactively, recognition and incentives for employees using The platform effectively, and publicizing success stories and resolution improvements.

Technical integration ensures APIs for integration with existing city-level systems, data export capabilities for external analysis and reporting, compliance with government data standards and regulations, and interoperability with state and national e-governance initiatives.

D. Comparative Advantage and Differentiation

CivicMind's competitive advantages over existing systems are substantial. Unlike Conventional e-governance platforms limited to digitization, CivicMind provides end-to-end AI automation from submission to resolution. The dual-model classification approach outperforms single- modality systems by 5-8% in accuracy while enabling fraud detection. The hybrid deduplication mechanism reduces redundant processing by 23% compared to rule-based approaches. Intelligent assignment balances workload and matches expertise, improving resolution times by 34-60% across urgency levels. Real-time transparency features build citizen trust through visibility and accountability.

Compared to international smart city platforms, CivicMind is specifically designed for Indian city-level context with consideration of infrastructure constraints, citizen

behavior patterns, and administrative workflows. The platform operates effectively on modest hardware requirements, unlike resource-intensive alternatives. The open architecture allows customization for different city-level authorities without extensive redevelopment. 000-75,000 implementation for medium city) is accessible for Indian city-level budgets.

E., The cost structure

Several promising directions exist for future enhancement. Multilingual NLP support through models like mBERT or XLM-R could enable processing of Hindi, Marathi, Tamil, and other Indian languages. Incorporating voice-based complaint submission would enhance accessibility for low-literacy populations.

Advanced fraud detection could leverage reverse image search integration to identify recycled images, EXIF metadata analysis for timestamp and location verification, and adversarial training to identify AI-generated or manipulated images.

Predictive analytics capabilities could forecast complaint hotspots based on historical patterns, predict infrastructure failure before citizen complaints occur, and optimize preventive maintenance scheduling to reduce reactive complaints.

Enhanced citizen engagement through mobile applications (Android and iOS) with offline capability, chatbot interface for natural language complaint submission and tracking, and social media integration for complaint submission via Twitter/Facebook would expand reach.

Integration with smart city sensors could enable IoT sensor data validation of complaints (e.g., water quality sensors confirming water supply issues), automatic complaint generation from sensor anomalies, and real-time infrastructure monitoring dashboards.

Blockchain integration for immutable audit trails could increase transparency and accountability, enable smart contracts for automated complaint routing and SLA enforcement, and provide cryptographic proof of complaint submission and resolution.

VII. CONCLUSION AND FUTURE WORK

This paper presented CivicMind, a comprehensive AI-enabled city-level complaint redressal system that addresses critical inefficiencies in Conventional complaint management through intelligent automation. The platform's dual-model architecture combining LSTM-based NLP for textual analysis with ResNet-50 CNN for image classification achieves classification accuracies exceeding 92% and 89% respectively, significantly outperforming baseline approaches. The cross-modal fraud detection

mechanism successfully identifies 87.3% of fraudulent submissions while maintaining low false positive rates, protecting city-level resources from misuse.

The hybrid geospatial-semantic deduplication algorithm reduces redundant complaint processing by 23%, freeing administrative capacity for genuine issues. The intelligent auto- assignment system balances employee workload while matching expertise to complaint types, resulting in 34-60% improvements in resolution times across urgency levels. Real-time transparency features transform citizen engagement from passive submission to active participation with visibility and accountability.

Experimental evaluation on real-world data and a 3-month pilot deployment with Thane city-level Corporation validate The platform's effectiveness. Citizen satisfaction ratings of 4.2/5.0 and employee satisfaction of 4.1/5.0 demonstrate acceptance from key stakeholders. Quantitative metrics show 34% reduction in average resolution time, 18% improvement in employee productivity, and estimated annual savings of ₹3.2 million for a medium-sized municipality.

The three-tier architecture with React frontend, Node.js backend, and FastAPI ML service demonstrates production-ready engineering suitable for large-scale deployment. Comprehensive security implementation, horizontal scaling capabilities, and extensive monitoring guarantee operational reliability and data protection. The platform processes complaints end-to-end in 2-5 seconds under normal load and scales gracefully to handle 100 complaints per minute with 500 concurrent users.

CivicMind contributes to the broader vision of smart cities and digital governance by demonstrating how AI can transform city-level operations beyond simple digitization. The platform's modular architecture and open design enable customization for different city-level contexts and integration with existing e-governance initiatives. The approach is replicable across Indian city-level authorities and potentially adaptable to other developing nations facing similar urbanization challenges.

Future work will focus on several key enhancements. Multilingual support through advanced NLP models will enable accessibility for non-English speaking populations, critical for widespread adoption in India. Mobile application development for Android and iOS platforms will enhance citizen convenience and enable offline complaint submission in areas with poor connectivity. Predictive analytics capabilities leveraging historical complaint data will enable proactive infrastructure maintenance, shifting from reactive to preventive governance. Integration with IoT sensors for smart city infrastructure will provide automated validation of complaints and early detection of issues before citizen reports. Blockchain-based immutable audit trails will further enhance transparency and accountability, building public trust in city-level institutions.

The success of CivicMind demonstrates that AI can be a powerful enabler of efficient, transparent, and citizen-centric governance. detecting fraud and duplicates, optimizing resource allocation, and providing real-time transparency, The platform addresses fundamental challenges facing rapidly urbanizing city-level authorities., By automating human-driven procedures As smart city initiatives continue to evolve globally, CivicMind represents a practical, deployable solution that bridges the gap between technological capability and real- world city-level needs.

In conclusion, CivicMind offers a comprehensive, production-ready platform for city-level complaint management that significantly improves upon Conventional systems through intelligent automation. The experimental results validate the technical approach, while pilot deployment demonstrates real-world viability. systems like CivicMind can play a crucial role in transforming urban governance and improving quality of life for millions of citizens in rapidly growing cities.

ACKNOWLEDGMENTS

This research was made possible through the use of publicly available civic datasets and open-source repositories. The authors are deeply grateful to Prof. Radhika B. for her mentorship and for overseeing the experimental validation of the CivicMind framework.

REFERENCES

- [1] Verma et al. (2023): Discussed in Section II-A regarding a PHP/MySQL-based Smart City complaint platform.
- [2] Singh et al. (2024): Cited in Section II-B for an AI-based system using NLP for automatic categorization.
- **[3] Patel and Joshi (2023):** Mentioned in Section II-A as developers of a mobile-based civic reporting application.
- [4] Khan et al. (2024): Referenced in Section II-E regarding an automated complaint routing system using rule-based engines.
- [5] Brown et al. (2022): Discussed in Section II-C for applying CNNs to detect urban infrastructure issues.
- [6] Zhang et al. (2024): Cited in Section II-C for developing a mobile application using MobileNet for real-time reporting.
- [7] Ahmed et al. (2023): Referenced in Section II-D for research on text similarity-based duplicate detection.
- **[8] Kumar and Sharma (2023):** Mentioned in Section II-B for exploring LSTM networks for complaint urgency prediction.